

Scalable Multi-Objective and Meta Reinforcement Learning via Gradient Estimation

Zhenshuo Zhang, Minxuan Duan, Youran Ye, Hongyang R. Zhang

Northeastern University, Boston, Massachusetts
{zhang.zhens, duan.mi, ye.you, ho.zhang}@northeastern.edu

Abstract

We study the problem of efficiently estimating policies that simultaneously optimize multiple objectives in reinforcement learning (RL). Given n objectives (or training tasks), we search for the optimal partition of n objectives into $k \ll n$ groups, with each group containing related objectives that can be trained together. This problem arises in applications such as robotics, control, and preference optimization in language models, where learning a single policy for all n objectives is suboptimal as n grows. We introduce a two-stage procedure—meta-training followed by fine-tuning—to address this problem. We first learn a meta-policy for all objectives using multitask learning. Then, we adapt the meta-policy to multiple randomly sampled subsets of objectives. The adaptation step leverages a first-order approximation property of well-trained policy networks, which is empirically verified to be accurate within a 2% error margin across various RL environments. The resulting algorithm, Policy-GradEx, efficiently estimates an aggregate task-affinity score matrix given a policy evaluation algorithm. Based on the estimated affinity matrix, we cluster the n objectives into k groups. Experiments on three robotic control benchmarks and the Meta-World benchmark demonstrate that our approach outperforms state-of-the-art baselines by 16% on average, while delivering up to $26\times$ faster speedup relative to performing full training to obtain the clusters. Ablation studies validate each component of our approach. For instance, compared with random grouping and gradient-similarity based grouping, our loss-based clustering approach yields an improvement of 19%. Finally, we analyze the generalization error of policy networks via measuring the Hessian trace of the loss surface and find that this gives non-vacuous measurements relative to the observed generalization errors.

Introduction

Reinforcement learning (RL) is a technique for sequential decision-making in interactive environments, enabling agents to learn from feedback signals. A central challenge in RL is to develop agents that generalize across a wide range of tasks rather than solving a single task in isolation. For instance, a robot needs to master multiple related skills that share an underlying structure (Yu et al. 2020b). Prior work has studied this problem for two closely related settings. The first, multitask reinforcement learning, seeks to

find a single policy that performs well across a given set of objectives (or tasks) (Sodhani, Zhang, and Pineau 2021; Joshi et al. 2025). The second, meta-reinforcement learning, aims to learn a meta-initialization that can rapidly adapt to unseen tasks (Finn, Abbeel, and Levine 2017; Rakelly et al. 2019). In both settings, however, the computational cost of evaluating and adapting a shared policy over many competing objectives grows quickly with n .

This paper addresses the problem of designing a (scalable) multi-objective optimizer for reinforcement learning, with natural extensions to policy adaptation. Learning from a diverse set of tasks has been shown to improve generalization (Thrun and Pratt 1998; Cobbe et al. 2019; Yin et al. 2020). However, the main difficulty is modeling task relationships and how different tasks transfer information to one another when trained together in a complex network (Wu, Zhang, and Ré 2020). Existing approaches have considered measuring the similarity between gradient vectors across tasks to quantify task relationships (Yu et al. 2020a). Notably, this gradient-similarity-based measure corresponds to pairwise task-affinity scores, whereas in practice it is natural to model how multiple policies interact when trained together (Li, Nguyen, and Zhang 2023). Exhaustively searching over all subsets of $\{1, 2, \dots, n\}$ is computationally infeasible: evaluating each subset typically requires a full training procedure, and the number of subsets grows exponentially in n (Caruana 1997). Greedy stepwise selection requires evaluation on $O(n^2)$ subsets, which is again impractical for large n (Li et al. 2023; Li, Sharma, and Zhang 2024).

To overcome these challenges, we introduce a scalable algorithm for estimating policy performance on arbitrary task subsets without full training. Our algorithm starts by learning a single meta-policy across all tasks, then uses a first-order surrogate model—derived from a Taylor approximation around the meta-policy—to efficiently estimate the outcome of fine-tuning on any subset. This enables us to compute a pairwise task affinity matrix and partition the n objectives into k subgroups using a convex clustering procedure. The resulting groups can be trained separately using any multitask or meta-RL optimizers. This overall approach is illustrated in Figure 1.

We validate our approach through extensive experiments on both Meta-World (Yu et al. 2020b) and robotic control benchmarks (Towers et al. 2025). We find that the first-

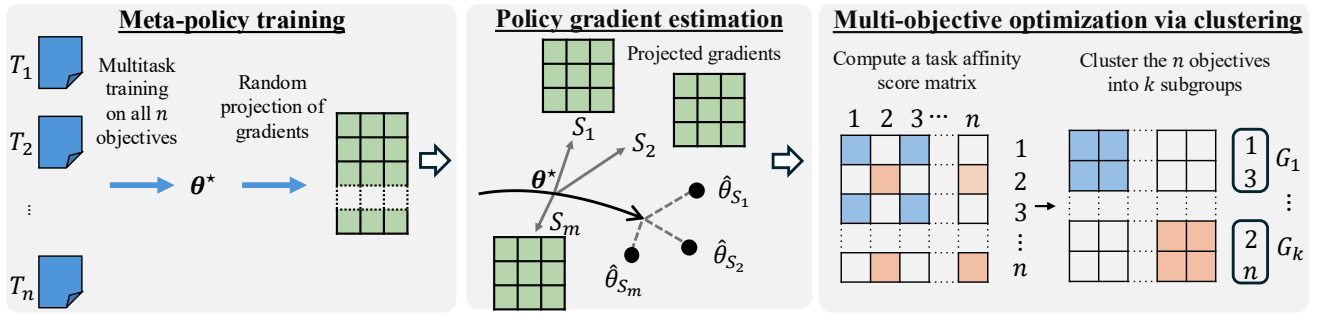


Figure 1: An overview of our approach. Left: Perform multitask training on all the objectives, leading to a meta-initialization policy θ^* . We store (randomly) projected gradients of θ^* for every transition. Middle: Estimate policy adaptation performance on m task subsets using projected gradients as features in logistic regression. Right: Use the estimation results to compute an n by n task affinity score matrix. Finally, we apply a grouping procedure to cluster similar objectives into one group, resulting in k subgroups.

order approximation applied to the outputs of a policy network yields accurate estimates—typically within 2% error margin—while being up to $26\times$ faster than full training. Our algorithm consistently outperforms multitask and meta-RL baselines (Yang et al. 2020; Sodhani, Zhang, and Pineau 2021; Sun et al. 2022) by 16% on average, and surpasses baseline grouping strategies (such as random grouping or gradient-similarity based grouping) by 23% and 16%, respectively. These results highlight the effectiveness and scalability of our task-affinity estimation approach.

Lastly, we conduct a theoretical analysis of the generalization errors of policy networks. The main technical approach is to compute (and measure) the trace of the Hessian of the loss surface (Ju, Li, and Zhang 2022; Ju et al. 2023; Zhang, Li, and Ju 2024). See Theorem 1 for the precise statement. When evaluated on the above RL environments, we find that the Hessian-based generalization bounds match the scale of empirical generalization errors (see Figure 2).

In summary, the contributions of this paper are threefold. (1) We propose a first-order gradient estimation algorithm to estimate policy performance on any given task subset efficiently. Based on this gradient estimation, we design a scalable algorithm that computes an $n \times n$ task affinity score matrix and partitions n objectives into k clusters via convex relaxation. (2) We conduct extensive experiments to validate the efficiency of our approach for several multi-objective and meta reinforcement learning benchmarks. The code for reproducing our results can be found online at <https://github.com/VirtuosoResearch/PolicyGradEx>. (3) We show theoretical bounds on the generalization error of policy networks by measuring the Hessian trace of the loss surface.

Problem Setup

We consider a standard reinforcement learning (RL) setting in which an agent must master a collection of related tasks. Let $\mathcal{T} = \{T_1, T_2, \dots, T_n\}$ denote the set of n source tasks. Each task T_i is modeled as a Markov decision process (MDP) $T_i = (\mathcal{S}, \mathcal{A}, P, P_0, r_i, \gamma)$, where all tasks share the same state space $\mathcal{S}$, action space $\mathcal{A}$, transition model P , and initial-state distribution P_0 , discount factor γ , but differ in

their reward functions r_i . For a policy π_θ with parameters θ , we define its task-specific expected reward as

$$R_i(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_i(s_t, a_t) \right].$$

In multi-objective RL, the goal is to learn a single policy that performs well across all tasks. A natural aggregate objective is the average reward

$$R_{\mathcal{T}}(\theta) = \frac{1}{n} \sum_{i=1}^n R_i(\theta).$$

More generally, for any subset $S \subseteq \mathcal{T}$, we define

$$R_S(\theta) = \frac{1}{|S|} \sum_{T_i \in S} R_i(\theta).$$

Pairwise task interactions can be characterized through this notion. For example, $R_{\{i,j\}}(\theta)$ captures the joint performance on tasks T_i and T_j , providing a meaningful measure of their affinity. A key challenge in multitask learning is *negative transfer*, where conflicting gradients from two tasks degrade optimization efficiency and final performance (Wu, Zhang, and Ré 2020). This occurs when

$$R_{\{i,j\}}(\theta) < R_i(\theta) \quad \text{or} \quad R_{\{i,j\}}(\theta) < R_j(\theta),$$

indicating that training on the pair together is detrimental compared to learning on a single task.

Our goal is to uncover the underlying affinity structure among the n tasks, which can be used to group similar tasks or select a small representative subset for meta-learning. However, searching for this structure requires evaluating the reward $R_S(\theta)$ for many different subsets S .

However, the number of subsets is 2^n , and even greedy selection strategies require evaluating $\mathcal{O}(n^2)$ subsets, where each evaluation typically involves a full RL training or adaptation procedure. This quickly becomes computationally prohibitive as n grows.

To address this challenge, we introduce **an efficient gradient-estimation algorithm to approximate the loss of any subset trained on a policy network without repeated training**. This enables us to scale up multi-objective RL to a large number of objectives n , as we will describe next.

Our Approach

This section describes our proposed algorithm for multi-objective reinforcement learning. The algorithm consists of two main stages. We first find a meta-policy θ^* trained using all tasks. Then, compute a surrogate model to estimate the adapted performance of θ^* on a subset of $\{1, 2, \dots, n\}$. This surrogate modeling framework enables efficient estimation of subset combinations without repeated training. In the second stage, we estimate the fine-tuning performance R_{S_i} on m randomly chosen subsets $S_1, S_2, \dots, S_m$ of $\mathcal{T}$, for every $1 \leq i \leq m$. For each subset, we solve a weighted logistic regression problem to estimate the adaptation parameters $\hat{\theta}_{S_i}$ and then estimate an approximate loss. Importantly, we run this second step using precomputed gradients and functional values at θ^* from the first stage. The key technique that enables this gradient estimation is a **first-order approximation property of the policy reward** that we also empirically verify for various RL settings.

Surrogate Modeling

Our goal is to estimate the output of a policy update starting from the meta-initialization θ^* . To do so efficiently, we linearize the policy gradient objective around θ^* and reformulate the one-step update as a weighted logistic regression problem.

We begin with the standard policy gradient objective used in algorithms such as Proximal Policy Optimization (PPO) (Schulman et al. 2017):

$$J(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right], \text{ where } r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta^*}(a_t|s_t)}$$

is the probability ratio and $\hat{A}_t$ is the estimated reward for the state-action pair (s_t, a_t) at time t . We perform a first-order Taylor expansion of the log-probability $\log \pi_\theta(a_t|s_t)$ around the initial parameters θ^* . Let $\Delta\theta = \theta - \theta^*$ and define the gradient feature vector

$$g_t = \nabla \log \pi_\theta(a_t|s_t)|_{\theta=\theta^*}.$$

The first-order Taylor expansion of the above log-probability-ratio is given by:

$$\begin{aligned} \log r_t(\theta) &= \log \pi_\theta(a_t|s_t) - \log \pi_{\theta^*}(a_t|s_t) \\ &= g_t^\top \Delta\theta + \epsilon, \end{aligned} \quad (1)$$

where ϵ refers to the approximation error of the expansion. Provided that $\Delta\theta$ is small relative to θ^* , we have another approximation as (with an error of $(1 - r_t)^2/2$)

$$r_t(\theta) \approx 1 + \log r_t(\theta). \quad (2)$$

By applying equations (1) and (2) back to $J(\theta)$, we have:

$$J(\theta) \approx \mathbb{E}_t \left[\hat{A}_t (1 + g_t^\top \Delta\theta) \right].$$

Since $\mathbb{E}_t[\hat{A}_t]$ does not depend on $\Delta\theta$, maximizing $J(\theta)$ reduces to maximizing the following which is linear in θ :

$$\mathbb{E}_t[\hat{A}_t g_t^\top (\theta - \theta^*)].$$

Surrogate loss. Next, we turn the above reward maximization on $J(\theta)$ into a weighted binary classification problem. For every $(s_t, a_t, \hat{A}_t)$, for any $t \in \{1, 2, \dots, N\}$, define:

Algorithm 1: POLICYGRADEX

Input: m subsets of n objectives $\mathcal{T}, S_1, S_2, \dots, S_m$

Require: Initial meta-policy parameter θ^* ; Number of episodes N ; Projection dimension d

```

1:  $\pi_{\theta^*} \leftarrow$  An initial policy trained on  $\mathcal{T}$ 
2:  $S_1, S_2, \dots, S_m \leftarrow m$  random subsets of  $\mathcal{T}$  with size  $\alpha$ 
3:  $P \leftarrow A$  by  $d$  isotropic Gaussian random projection matrix
4: for  $t = 1, \dots, N$  do
5:    $g_{i,t} \leftarrow P^\top \nabla \pi_{\theta^*}(a_t|s_t), \forall T_i \in \mathcal{T}$ 
6:    $w_{i,t} \leftarrow |\hat{A}_t|, \forall T_i \in \mathcal{T}$ 
7:    $y_{i,t} \leftarrow \text{sign}(\hat{A}_t), \forall T_i \in \mathcal{T}$ 
8: end for
9: for  $j = 1, \dots, m$  do
10:   $\hat{\Delta\theta}_d \leftarrow \arg \min_{\Delta\theta \in \mathbb{R}^d} \hat{L}_{S_j}(\theta^* + P\Delta\theta)$ 
11:   $\hat{\theta}_j \leftarrow \theta^* + P\hat{\Delta\theta}_d$ 
12:   $\hat{f}(S_j) \leftarrow -\hat{L}_{S_j}(\hat{\theta}_j)$ 
13: end for
14: Return  $\hat{f}(S_1), \dots, \hat{f}(S_m)$ 
```

- Target binary label $y_t = \text{sign}(\hat{A}_t) \in \{-1, +1\}$, indicating if the action was better or worse than zero.
- Classifier score $z_t = g_t^\top \Delta\theta$, linear score for the update.
- Sample weight $w_t = |\hat{A}_t|$, magnitude of the reward.

This yields a per-sample surrogate loss as follows:

$$\ell(g_t, y_t, w_t; \Delta\theta) = w_t \cdot \log(1 + (-y_t(g_t^\top \Delta\theta))).$$

For a given task subset S , let $\mathcal{D}_S$ denote the set of trajectory samples $(g_{i,t}, y_{i,t}, w_{i,t})$ from all the tasks $T_i \in S$, across all the samples (with an extra index i). The average loss for the subset S is defined by the average loss over all the samples from all the tasks within the subset S :

$$\hat{L}_S(\theta) = \frac{1}{|\mathcal{D}_S|} \sum_{(g,y,w) \in \mathcal{D}_S} \ell(g, y, w; \theta). \quad (3)$$

Minimizing this loss over $\Delta\theta = \theta - \theta^*$ yields the estimated update for the subset S on top of the meta-policy θ^* .

Random projection. Since the gradient vectors $g_{i,t} \in \mathbb{R}^p$ may be high-dimensional (with p in the order of millions), we apply random projections to reduce the dimension down to a few hundred (in practice, $d = 400$ suffices). This projection provably preserves the pairwise similarity between all pairs of gradient vectors via the Johnson–Lindenstrauss Lemma (Johnson and Lindenstrauss 1984). We create a projection matrix $P \in \mathbb{R}^{p \times d}$, where $d \ll p$ and entries are independently sampled from $\mathcal{N}(0, d^{-1})$. We project the high-dimensional gradients to a low-dimensional space

$$\tilde{g}_{i,t} = P^\top g_{i,t}.$$

Then, we solve the logistic regression problem in the reduced space $\mathbb{R}^d$:

$$\hat{\Delta\theta}_d \leftarrow \arg \min_{\Delta\theta_d \in \mathbb{R}^d} \frac{1}{|\mathcal{D}_S|} \sum_{\mathcal{D}_S} \ell(\tilde{g}_{i,t}, y_{i,t}, w_{i,t}; \Delta\theta_d). \quad (4)$$

Distance	MT10	CartPole	Highway	Lunarlander
0.1%	0.01 $\pm$ 0.01%	0.12 $\pm$ 0.14%	0.02 $\pm$ 0.02%	0.06 $\pm$ 0.01%
0.5%	0.43 $\pm$ 0.73%	0.73 $\pm$ 0.10%	0.11 $\pm$ 0.09%	0.03 $\pm$ 0.02%
1.0%	0.32 $\pm$ 0.56%	0.98 $\pm$ 0.65%	2.04 $\pm$ 0.58%	0.48 $\pm$ 0.01%

Table 1: We empirically find that the approximation error of ϵ is negligible on several interactive RL environments. We report the mean and standard deviation of the approximation error across 10 random subsets. We provide a detailed description of the setup in the experiments section.

Finally, we turn the solution from dimension d back to the original dimension p as

$$\hat{\theta}_S = P\hat{\theta}_d + \theta^*.$$

The complete description of this surrogate modeling procedure is provided in Algorithm 1.

Evaluation of First-Order Policy Approximation

A critical assumption is the accuracy of the approximation in equation (1). Here we observe that the approximation error remains negligible for θ around θ^* . We evaluate this approximation by measuring the relative residual sum of squares (RSS) error across multiple RL environments. Specifically, we compute:

$$\frac{(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta^*}(a_t|s_t) - g_t^\top \Delta\theta)^2}{(\log \pi_\theta(a_t|s_t))^2},$$

where $\Delta\theta = \theta - \theta^*$ and $g_t = \nabla \log \pi_\theta(a_t|s_t)|_{\theta=\theta^*}$.

We begin by verifying that adapted policies remain close to the meta-initialization θ^* . Using ten randomly sampled subsets, we evaluate the relative distance $\frac{\|\theta - \theta^*\|_F}{\|\theta^*\|_F}$, confirming that adaptation stays within a local neighborhood where the approximation is valid.

We report the RSS error measured on MT10 from the MetaWorld benchmark, CartPole, LunarLander, and Highway in Table 1. In these environments, the policy, a four-layer MLP, is trained to perform various control tasks by selecting an action based on its output given the states. We sample 2048 steps per task to obtain the gradients. The results are averaged over 10 randomly sampled subsets of size 5. For a proper initialization θ^* , the approximation error is less than 2% when the updated policy θ remains close to the initial policy. This verifies that the first-order expansion provides a sufficiently accurate local model for surrogate-based policy estimation. The approximation error increases from less than 2% to up to 10% when the policy’s parameter distance from initialization grows to around 5%, defining the boundaries of our method’s applicability.

Task Affinity Grouping

We next quantify the pairwise relationships between tasks by constructing a task affinity matrix $U \in \mathbb{R}^{n \times n}$, where each entry $U_{i,j}$ captures the collaborative behavior between tasks T_i and T_j . Larger values indicate that jointly fine-tuning on the two tasks produces higher surrogate performance.

Algorithm 2: Clustering related objectives into subgroups using the task affinity score matrix

Input: n tasks $\mathcal{T}$; number of desired clusters k

Require: Number of subsets m and size α , rounding threshold λ ; Number of episodes N , projection dimension d

Output: A disjoint partition of $[n]$ as C

- 1: $\theta^* \leftarrow$ Train a meta-initialization policy with $\mathcal{T}$
- 2: $S_1, \dots, S_m \leftarrow$ Sample m subsets of size α from $\mathcal{T}$
- 3: $\hat{f}(S_1), \hat{f}(S_2), \dots, \hat{f}(S_m) \leftarrow$ Apply POLICYGRADEX with $(S_1, S_2, \dots, S_m; \theta^*; N, d)$
- 4: $U \leftarrow$ An $n \times n$ affinity score matrix via equation (5)
- 5: $X \leftarrow$ Solve a convex relaxation program (7) with U and regularization parameter λ
- 6: $G_1, G_2, \dots, G_k \leftarrow$ Round X into k subgroups

A naive approach is to repeatedly train a model for all the $\binom{n}{2}$ subsets, which is impractical. Instead, we sample m random subsets $\{S_1, \dots, S_m\}$ and use the surrogate model to compute their estimated performance $\hat{f}(S_i) = -\hat{L}_{S_i}(\hat{\theta}_i)$, for all $i = 1, 2, \dots, m$ based on Algorithm 1. We then compute an $n \times n$ task affinity score matrix as follows:

$$U_{i,j} = \frac{1}{n_{i,j}} \sum_{1 \leq l \leq m: \{T_i, T_j\} \in S_l} \hat{f}(S_l), \quad (5)$$

for all i and j in $\{1, 2, \dots, n\}$, where $n_{i,j}$ is the number of sampled subsets containing both T_i and T_j . As a remark, provided that $m = O(n^2)$, then $n_{i,j}$ must be nonzero with high probability, for all i and j in $\{1, 2, \dots, n\}$.

We then cluster tasks by solving a convex optimization problem that maximizes intra-cluster affinity, following a trace-regularized relaxation (Awasthi et al. 2015). This step is very fast since it runs on an n by n matrix with n being at most several hundred, which can typically be solved in just a few seconds. Moreover, this procedure can be repeated with different values of λ to determine the optimal number of clusters k for downstream performance. For brevity, we defer a complete description of the convex relaxation program to the Appendix. The whole procedure is described in Algorithm 2.

Experiments

We conduct a comprehensive set of experiments to validate POLICYGRADEX and its application across various multi-objective reinforcement learning benchmarks. Our evaluation is designed to answer two key questions: (1) Does our surrogate model accurately and efficiently approximate the outcome of full policy training on various task subsets? (2) Do task groups identified by our algorithm lead to superior performance in downstream multitask and meta-RL evaluations compared to state-of-the-art and heuristic baselines?

Our finding shows that our surrogate modeling approach achieves high accuracy in estimating multitask RL performance, with over 0.73 normalized mutual information relative to the ground-truth, while reducing the number of floating-point operations (FLOPs) by up to 26 $\times$. In downstream evaluations, POLICYGRADEX outperforms existing

Environment	Meta-World	CartPole	Highway	LunarLander
Multi-task training (Yu et al. 2020b)	71.3 $\pm$ 1.2%	145.9 $\pm$ 9.0	140.0 $\pm$ 4.6	53.8 $\pm$ 14.6
Soft modularization (Yang et al. 2020)	82.0 $\pm$ 1.1%	139.3 $\pm$ 9.5	141.3 $\pm$ 3.5	66.1 $\pm$ 13.0
PaCo (Sun et al. 2022)	73.1 $\pm$ 1.1%	144.5 $\pm$ 5.2	136.6 $\pm$ 6.4	62.6 $\pm$ 11.4
CARE (Sodhani, Zhang, and Pineau 2021)	84.0 $\pm$ 1.8%	/	/	/
Random grouping	58.2 $\pm$ 6.2%	144.1 $\pm$ 10.1	143.4 $\pm$ 5.4	73.1 $\pm$ 11.7
Gradient similarity-based grouping (Yu et al. 2020a)	69.6 $\pm$ 1.9%	142.0 $\pm$ 8.2	135.6 $\pm$ 7.6	80.8 $\pm$ 6.9
Algorithm 2	94.0$\pm$2.8%	159.2$\pm$3.8	153.5$\pm$7.8	82.8$\pm$6.9

Table 2: We report the average success rate on the Meta-World benchmark and the rewards after adaptation for three control environments. We note that CARE does not apply to the control tasks because of the lack of contextual information. We report the mean and standard deviation from five runs.

multitask optimizers by 19% in multitask RL benchmarks and achieves a 13% improvement in the meta-RL setting. Furthermore, we compare our task affinity grouping algorithm to random grouping and gradient-similarity based grouping, showing that POLICYGRADEX achieves a 19% improvement over both. Ablation analysis validates the use of random projections and the choice of k in the algorithm.

Experimental Setup

Environments. We evaluate our method on two types of benchmarks. First, we use MT10 from the Meta-World benchmark (Yu et al. 2020b), which consists of 10 diverse robotic manipulation tasks. While these tasks share common state and action spaces, their reward functions and dynamics vary (e.g., a goal position in one task may correspond to an object’s position in another). Second, we use three classic control environments from Gymnasium (Towers et al. 2025) and MO-Gymnasium (Felten et al. 2023): CartPole, Highway, and LunarLander. For each, we generate a distribution of 10 source tasks by altering key physical parameters (e.g., pole length for CartPole, traffic density for Highway, and gravity for LunarLander).

Baselines. We compare our method against two categories of baselines. (1) Multi-objective RL baselines: These represent standard and state-of-the-art multitask approaches for multitasks training: a single policy trained on all tasks; Soft Modularization (Yang et al. 2020), which employs a routing-based selection mechanism; PaCo (Sun et al. 2022), an ensemble method that composes policies from a shared parameter subspace; and CARE (Sodhani, Zhang, and Pineau 2021), which performs inference using contextual information to adapt policies. (2) Grouping baselines: These methods use the same number of groups as our approach but employ different heuristic grouping procedures. This allows us to isolate the benefit of our affinity modeling. We include random grouping (where tasks are randomly assigned to k groups) and gradient-similarity-based grouping (where tasks are clustered based on the cosine similarity of their respective policy gradients).

Implementations. For the MT10 benchmark, we group the $n = 10$ tasks into $k = 3$ subsets and train a separate policy for each subgroup using soft modularization. We measure performance using the average success rate per task. For the control environments, we form the meta-training set

MLP layers	MetaWorld	LunarLander	Speedup
2	0.76	0.73	21 $\times$
4	0.76	0.73	24 $\times$
8	0.76	0.73	26 $\times$

Table 3: Normalized Mutual Information (NMI) between estimated and actual clusters, measured on two environments trained with a multi-layer perceptron (MLP). The speedup is measured as the ratio of the FLOP count between full training and POLICYGRADEX.

by randomly selecting one from each task group provided by POLICYGRADEX. We then use MAML (Finn, Abbeel, and Levine 2017) to train a meta-policy on the assigned tasks. The final performance is measured by the average reward after 200 adaptation steps on 50 unseen target tasks. All experiments are conducted using PyTorch on an Ubuntu server with an Intel Xeon E5-2623 CPU and an NVIDIA Quadro RTX 6000 GPU.

Results for Loss Function Estimation

We first evaluate both the approximation accuracy and computational cost of POLICYGRADEX. We establish ground-truth subset clusters based on task-affinity scores derived from rewards obtained by training policies on different task subsets. We then use Normalized Mutual Information (NMI) to quantify the accuracy of the subsets estimated by POLICYGRADEX against this ground truth.

As shown in Table 3, on both MetaWorld and LunarLander, POLICYGRADEX identifies subset clusters that achieve over 0.73 similarity to those obtained from full-policy training, while reducing FLOPs by a factor of 26 $\times$. By contrast, the NMI under random clustering is approximately 0.2. Moreover, the relative error between the estimated task affinity matrix and the ground-truth matrix is at most 0.2.

Results for Multi-Objective RL

We report the evaluation results in Table 2. On the Meta-World benchmark, our approach improves the average success rate by 21% compared to multitask optimizers. It also achieves a 62% improvement over training with random groups and a 35% improvement over training with gradient-similarity-based groups. This observation indicates that ran-

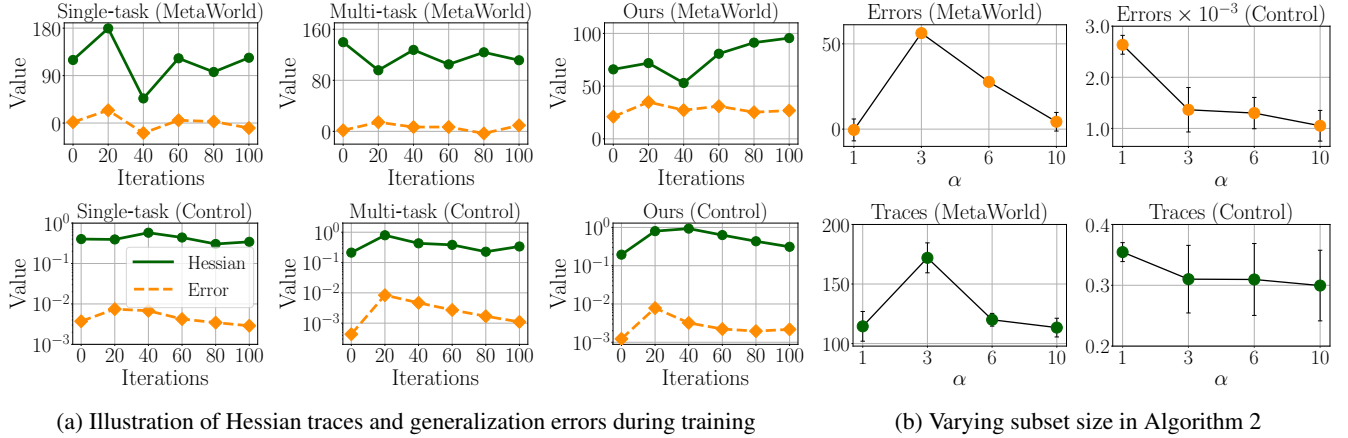


Figure 2: Illustrating the Hessian trace measurements and empirical generalization errors with respect to the policy network. Figure 2a: Showing that the Hessian trace is comparable in scale to the observed generalization errors, tested on Meta-World and a control task. Figure 2b: Showing that in a Meta-World environment, the generalization error reaches the highest when the subset size $\alpha = 3$, suggesting negative transfer among a small set of tasks. In the meta-RL control task, the generalization performance monotonically improves with the addition of more tasks in each group.

dom grouping and gradient similarity-based grouping in Meta-World tasks can lead to negative transfer. In contrast, POLICYGRADEX selects task groups with positive transferability, thereby improving the final success rate.

Next, we evaluate POLICYGRADEX’s ability to select a representative task subset for meta-learning in the three control RL environments. Using MAML as the meta-learner, our approach improves the final adapted reward by 7% compared to multitask optimizers.

Finally, we compare with two baseline meta-training settings: training on all available source tasks and training on randomly grouped tasks. Our approach achieves a 13% improvement over both baselines and a 9% improvement compared to gradient-similarity-based grouping.

Generalization Error Measurements via Hessians

Next, we analyze the generalization behavior by evaluating a sharpness measure based on the Hessian trace of the test loss for the policy network and the policy gradient loss. A smaller trace value suggests a flatter loss landscape. We implement Hutchinson’s estimator and leverage a faster version of this estimator (Meyer et al. 2021) to estimate the Hessian trace of RL models. We compute the policy gradient loss from the target tasks and measure the difference between the training loss and test loss as the generalization error. Results for the largest eigenvalue of the Hessian are similar.

First, we evaluate the Hessian trace and the policy generalization error between single-task training, multitask training on ten tasks, and training on a task group selected by POLICYGRADEX. We begin training from the initial policy for 100 iterations, each with 2048 steps.

Figure 2a shows the dynamics of the trace of the Hessian and the generalization errors during training. We find that they both follow a qualitatively similar trend. Next, as shown in Figure 2b, we find that single-task training yields a very low generalization error in the Meta-World bench-

mark, indicating that a single policy can solve an individual task. However, when the task count reaches three, the generalization error sharply increases, suggesting that a single policy struggles to perform well across multiple tasks. As the number of tasks increases, the generalization error and the Hessian trace decrease, indicating the positive transfer among tasks. In the control environments, we observe that both the generalization error and the Hessian trace decrease as α increases.

Ablation Analysis

Given that each task is solvable in the Meta-World benchmark, our goal is to determine the minimum number of groups k required to maximize the success rate. We vary k from 1 to 4, observing average success rates of 94.0% when $k = 3$, compared to 89.5% when $k = 2$ and 95.1% when $k = 4$. Thus, we report the final results with three groups. For the control environments, we vary k from 1 to 5. We find that the success rate stabilizes after k reaches 3, so we report results using three groups.

For the random projection dimension d , we vary it from 200 to 1000 and observe that values beyond 400 yield minimal gains, so we fix $d = 400$.

Generalization Error Analysis via Hessians

In this section, we present a generalization error analysis for the RL setting described above. We present a Hessian-based technique to quantify generalization errors, applicable to any kind of policy network used in RL training. We first describe the tools that we will use. Given a data distribution of $\mathcal{D}$, let

$$L(f_W) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [l(f_W(x), y)]$$

denote the expected loss of an input sample x, y . Let $\hat{L}(f_W)$ denote the empirical loss given n independent samples

$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ sampled from $\mathcal{D}$:

$$\hat{L}(f_W) = \frac{1}{n} \sum_{i=1}^n \ell(f_W(x_i), y_i).$$

Let $\mathcal{W} \subseteq \mathbb{R}^d$ denote the weight space of W . Assume that the weighted Hessian along x is bounded by a fixed constant $\mathcal{H}$ that does not grow with d or n :

$$\mathcal{H} := \sup_{W \in \mathcal{W}} \left(W^\top \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2 \ell(f_W(x), y)]^+ W \right),$$

where $[\nabla^2 \ell(f_W(x), y)]^+$ means that we truncate the negative eigen-directions of the Hessian to zero. Then, we have the following generalization error bound, which applies uniformly to the hypothesis space $\{f_W : W \in \mathcal{W}\}$.

Theorem 1. *Assume that the loss function ℓ is bounded between 0 and C for a fixed constant $C > 0$ on the data distribution $\mathcal{D}$. Suppose $\ell(f_W(\cdot), \cdot)$ is twice-differentiable in W and the Hessian matrix $\nabla^2 \ell(f_W(\cdot), \cdot)$ is Lipschitz-continuous within the weight space $\mathcal{W}$. Suppose there exists a fixed bound $\mathcal{H}$ on the Hessian trace over any $W \in \mathcal{W}$. Then, for any W in $\mathcal{W}$, any $\epsilon > 0$ small enough, and any small $\delta > 0$, with probability at least $1 - \delta$ over the randomness of n samples, we have:*

$$L(f_W) \leq (1 + \epsilon) \hat{L}(f_W) + (1 + \epsilon) \sqrt{\frac{C \cdot \mathcal{H}}{n}} + \epsilon, \quad (6)$$

where $\epsilon = O(n^{-3/4} \log(\delta^{-1}))$ denotes the error term.

The proof of Theorem 1 is based on a PAC-Bayes analysis (Ju, Li, and Zhang 2022; Zhang, Li, and Ju 2024), and the details are in the Appendix. In particular, a new aspect of this result is that we use an anisotropic perturbation in the prior and posterior distributions. We build on a linear PAC-Bayes bound (See Theorem 2 in the Appendix), and optimize the noise distribution to obtain the $\mathcal{H}$ measure.

In Figure 2, we visualize the Hessian trace on the Meta-World and the control tasks. Our results show that the Hessian trace is comparable in scale to the observed generalization errors. In the Meta-World benchmark, the generalization error increases as the number of tasks increases from one to three, suggesting negative transfer. As more tasks are added, the error decreases, indicating positive transfer.

Related Work

Multi-objective optimization for reinforcement learning. Early works on multitask learning use hard parameter sharing, where a single neural network backbone is shared among all tasks, with only the final layers or heads being task-specific (Yu et al. 2020b). A canonical example is the multi-head Soft Actor-Critic (SAC) architecture, which serves as a common baseline (Yu et al. 2020b).

The multitask objective provides implicit regularization across multiple tasks trained together in a shared network (Wu, Zhang, and Ré 2020), which can be formalized in a high-dimensional regression setting (Yang et al. 2025). Training the model to learn representations useful across a diverse set of tasks helps prevent overfitting. A deeper analysis of the regularization effect behind multitask RL is an interesting question for future work.

Meta-reinforcement learning. Meta reinforcement learning may be viewed as a specific scenario of meta learning. MAML (Finn, Abbeel, and Levine 2017) is a foundational work that introduces a bi-level optimization structure. In the inner loop, the policy is adapted to specific source tasks. In the outer loop, the initial meta-policies are updated by differentiating with respect to inner-loop adaptation, optimizing for the expected performance of the adapted policy across a distribution of source tasks. Building on this, ProMP (Rothfuss et al. 2019) introduces a low variance curvature estimator to stabilize training by assigning weight to each sample in trajectories. Tr-MAML (Collins, Mokhtari, and Shakkottai 2020) focuses on the most challenging task and introduces a min-max framework to minimize the maximum loss in the set. Further exploring gradient estimation (Li et al. 2025; Zhang et al. 2025) for designing ensemble methods in meta-RL is an interesting question for future work.

Data attribution and model interpretability. Another line of work involves data modeling, which aims to trace a model’s predictions back to its training data. One method is to retrain the model on a different subset of the dataset and estimate the Shapley value of each sample (Ghorbani and Zou 2019). A commonly used technique is influence functions (Koh and Liang 2017), which use Hessians to approximate the effect of removing a sample. Datamodels (Ilyas et al. 2022) finds that a linear regression method can accurately approximate the outcome of deep nets trained with a subset of samples on computer vision tasks. Li, Nguyen, and Zhang (2023) introduce a linear surrogate model for multitask learning and analyze the sample complexity of linear surrogate models. TRAK (Park et al. 2023) further demonstrates that the approximate solution of linear regression computed with projected gradients delivers comparable results to the original linear model for ImageNet, CLIP, and BERT. Our work contributes to this literature by applying attribution methods to design RL algorithms.

Conclusion

This paper introduces an efficient algorithm for multi-objective reinforcement learning. The overall approach works by first training a meta-policy via multitask learning. Then, use a gradient-estimation algorithm to adapt this meta-policy to multiple subsets of training objectives. The key observation is that a policy can effectively adapt across different task subsets through a first-order approximation with a proper initialization. Leveraging this, we propose a surrogate model to approximate the actual training performance of the policy, which enabled us to derive task affinities and subsequently cluster similar tasks into groups. In extensive reinforcement learning tasks, our method consistently improves performance. Lastly, we measure sharpness via Hessians to analyze generalization in multi-objective RL. We hope this work inspires further studies on designing principled methods for multi-objective reinforcement learning and quantifying generalization in policy learning algorithms. Further applying our approach to broader RL settings is another promising direction for future work.

Acknowledgments

Thanks to the anonymous referees for their constructive feedback. This work is partially funded by NSF award IIS-2412008. Any views or opinions expressed herein are solely those of the authors listed, and may differ from those expressed by NSF.

References

- Awasthi, P.; Bandeira, A. S.; Charikar, M.; Krishnaswamy, R.; Villar, S.; and Ward, R. 2015. Relax, no need to round: Integrality of clustering formulations. In *Innovations in Theoretical Computer Science*, 191–200.
- Caruana, R. 1997. Multitask learning. *Machine learning*, 28(1): 41–75.
- Chi, Y.; Chen, Y.; and Wei, Y. 2025. Statistical and Algorithmic Foundations of Reinforcement Learning. *arXiv preprint arXiv:2507.14444*.
- Cobbe, K.; Klimov, O.; Hesse, C.; Kim, T.; and Schulman, J. 2019. Quantifying generalization in reinforcement learning. In *International Conference on Machine Learning*, 1282–1289.
- Collins, L.; Mokhtari, A.; and Shakkottai, S. 2020. Task-robust model-agnostic meta-learning. *Advances in Neural Information Processing Systems*, 33: 18860–18871.
- Felten, F.; Alegre, L. N.; Nowé, A.; Bazzan, A. L. C.; Talbi, E. G.; Danoy, G.; and Silva, B. C. da. 2023. A Toolkit for Reliable Benchmarking and Research in Multi-Objective Reinforcement Learning. In *Neural Information Processing Systems*.
- Finn, C.; Abbeel, P.; and Levine, S. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*, 1126–1135.
- Foster, D. J.; and Rakhlin, A. 2023. Foundations of reinforcement learning and interactive decision making. *arXiv preprint arXiv:2312.16730*.
- Ghorbani, A.; and Zou, J. 2019. Data shapley: Equitable valuation of data for machine learning. In *International Conference on Machine Learning*, 2242–2251.
- Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning*, 1861–1870.
- Ilyas, A.; Park, S. M.; Engstrom, L.; Leclerc, G.; and Madry, A. 2022. Datamodels: Predicting predictions from training data. *International Conference on Machine Learning*.
- Johnson, W. B.; and Lindenstrauss, J. 1984. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary mathematics*, 26(189-206): 1.
- Joshi, V.; Xu, Z.; Liu, B.; Stone, P.; and Zhang, A. 2025. Benchmarking massively parallelized multi-task reinforcement learning for robotics tasks. *Reinforcement Learning Journal*.
- Ju, H.; Li, D.; Sharma, A.; and Zhang, H. R. 2023. Generalization in graph neural networks: Improved pac-bayesian bounds on graph diffusion. In *International conference on artificial intelligence and statistics*, 6314–6341.
- Ju, H.; Li, D.; and Zhang, H. R. 2022. Robust fine-tuning of deep neural networks with hessian-based generalization guarantees. In *International Conference on Machine Learning*, 10431–10461.
- Koh, P. W.; and Liang, P. 2017. Understanding black-box predictions via influence functions. In *International Conference on Machine Learning*, 1885–1894.
- Konda, V.; and Tsitsiklis, J. 1999. Actor-critic algorithms. *Advances in neural information processing systems*, 12.
- Li, D.; Ju, H.; Sharma, A.; and Zhang, H. R. 2023. Boosting multitask learning on graphs through higher-order task affinities. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1213–1222.
- Li, D.; Nguyen, H. L.; and Zhang, H. R. 2023. Identification of negative transfers in multitask learning using surrogate models. *Transactions on Machine Learning Research*.
- Li, D.; Sharma, A.; and Zhang, H. R. 2024. Scalable Multitask Learning Using Gradient-based Estimation of Task Affinity. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 1542–1553.
- Li, D.; Zhang, Z.; Wang, L.; and Zhang, H. R. 2025. Efficient Ensemble for Fine-tuning Language Models on Multiple Datasets. *Association for Computational Linguistics*.
- Liu, B.; Liu, X.; Jin, X.; Stone, P.; and Liu, Q. 2021. Conflict-averse gradient descent for multi-task learning. In *International Conference on Learning Representations*.
- McAllester, D. 2013. A PAC-Bayesian tutorial with a dropout bound. *arXiv preprint arXiv:1307.2118*.
- Meyer, R. A.; Musco, C.; Musco, C.; and Woodruff, D. P. 2021. Hutch++: Optimal stochastic trace estimation. In *Symposium on Simplicity in Algorithms (SOSA)*, 142–155. SIAM.
- Park, S. M.; Tsvetkov, Y.; Költer, J. Z.; and Salakhutdinov, R. 2023. Trak: Attributing model behavior at scale. In *International Conference on Machine Learning*, 27457–27481.
- Rakelly, K.; Zhou, A.; Finn, C.; Levine, S.; and Quillen, D. 2019. Efficient off-policy meta-reinforcement learning via probabilistic context variables. In *International Conference on Machine Learning*, 5331–5340.
- Rothfuss, J.; Lee, D.; Clavera, I.; Asfour, T.; and Abbeel, P. 2019. ProMP: Proximal Meta-Policy Search. In *International Conference on Learning Representations*.
- Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Sodhani, S.; Zhang, A.; and Pineau, J. 2021. Multi-task reinforcement learning with context-based representations. In *International Conference on Machine Learning*, 9767–9779.
- Sun, S.; Shi, H.; Zhao, Z.; Wang, X.; and Zhou, J. 2022. PaCo: Parameter-compositional multi-task reinforcement learning. In *International Conference on Learning Representations*.

- Teh, Y. W.; Bapst, V.; Czarnecki, W. M.; Quan, J.; Kirkpatrick, J.; Hadsell, R.; Heess, N.; and Pascanu, R. 2017. Distral: Robust multitask reinforcement learning. In *Advances in neural information processing systems*, volume 30.
- Thrun, S.; and Pratt, L. 1998. Learning to learn. Springer.
- Towers, M.; Kwiatkowski, A.; Terry, J.; Balis, J. U.; De Cola, G.; Deleu, T.; Goulão, M.; Kallinteris, A.; Krimmel, M.; KG, A.; Perez-Vicente, R.; Pierré, A.; Schulhoff, S.; Tai, J. J.; Tan, H.; and Younis, O. G. 2025. Gymnasium: A standard interface for reinforcement learning environments. *NeurIPS Datasets and Benchmarks*.
- Vershynin, R. 2018. *High-dimensional probability: An introduction with applications in data science*, volume 47. Cambridge university press.
- Wainwright, M. J. 2019a. *High-Dimensional Statistics: A Non-Symptotic Viewpoint*. Cambridge University Press.
- Wainwright, M. J. 2019b. Variance-reduced Q -learning is minimax optimal. *arXiv preprint arXiv:1906.04697*.
- Wu, S.; Zhang, H. R.; and Ré, C. 2020. Understanding and Improving Information Transfer in Multi-Task Learning. In *International Conference on Learning Representations*.
- Yang, F.; Zhang, H. R.; Wu, S.; Re, C.; and Su, W. J. 2025. Precise High-Dimensional Asymptotics for Quantifying Heterogeneous Transfers. *Journal of Machine Learning Research*, 26(113): 1–88.
- Yang, R.; Xu, H.; Wu, Y.; and Wang, X. 2020. Multi-task reinforcement learning with soft modularization. *Advances in Neural Information Processing Systems*, 33: 4767–4777.
- Yin, M.; Tucker, G.; Zhou, M.; Levine, S.; and Finn, C. 2020. Meta-learning without memorization. *International Conference on Learning Representations*.
- Yu, T.; Kumar, S.; Gupta, A.; Levine, S.; Hausman, K.; and Finn, C. 2020a. Gradient surgery for multi-task learning. In *Advances in neural information processing systems*, volume 33, 5822–5833.
- Yu, T.; Quillen, D.; He, Z.; Julian, R.; Hausman, K.; Finn, C.; and Levine, S. 2020b. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on Robot Learning*, 1094–1100.
- Zhang, H. R.; Li, D.; and Ju, H. 2024. Noise Stability Optimization for Finding Flat Minima: A Hessian-based Regularization Approach. *Transactions on Machine Learning Research*.
- Zhang, Z.; Zhang, Z.; Li, D.; Wang, L.; Dy, J.; and Zhang, H. R. 2025. Linear-Time Demonstration Selection for In-Context Learning via Gradient Estimation. In *Conference on Empirical Methods in Natural Language Processing*, 16470–16488.

Clustering Algorithms

We provide further details on using optimization to cluster task groups based on the task affinity matrix. For a partition into m clusters, represented by binary indicator vectors $v_1, \dots, v_m$, the objective, which corresponds to the average density of the m clusters, is given by

$$\sum_{i=1}^m \frac{v_i^\top U v_i}{v_i^\top v_i}.$$

However, directly optimizing this combinatorial objective is NP-hard. We therefore introduce a matrix variable

$$X = \sum_{i=1}^m \frac{v_i v_i^\top}{v_i^\top v_i}.$$

The objective can then be rewritten as maximizing the inner product $\langle U, X \rangle$. In this formulation, the number of clusters m is equal to both the rank of X . Relaxing the non-convex rank constraint leads to a semi-definite program (SDP) formulation. To avoid specifying the number of clusters m and to automatically determine a suitable number of groups, we penalize the trace of X in the objective. This yields the following regularized optimization problem:

$$\begin{aligned} \max_{X \in \mathbb{R}^{n \times n}} \quad & \langle U, X \rangle - \lambda \cdot \text{Tr}(X) \\ \text{s.t.} \quad & X e = e, \text{Tr}[X] = k \\ & X \succeq 0, X \geq 0, \end{aligned} \tag{7}$$

where e is the vector of all ones, and $\lambda > 0$ is a hyperparameter that controls the penalty on the number of clusters, effectively balancing clustering quality with the total number of groups.

Once we solve the SDP for the optimal X , we recover the discrete cluster assignments through a rounding procedure. This clustering procedure is efficient to implement in practice; solving the SDP for a typical number of tasks takes several seconds. Thus, the computational cost is negligible compared to the cost of model training.

Proof of Theorem 1

We provide a high-level illustration of the proof of Theorem 1. There are two main steps in the proof. First, we would like to show that on the original input distribution, the gap between test and training losses remains bounded by the Hessian distance measure. Second, we show that such a statement holds for every transformed distribution of the input distribution. Thus, we reduce the proof of equation (6) to the first step.

Proof sketch. We start with the first step. Let

$$\begin{aligned} \hat{L}(f_W) &= \frac{1}{n} \sum_{i=1}^n \ell(f_W(x_i), y_i), \text{ and} \\ L(f_W) &= \mathbb{E}_{(x,y) \sim \mathcal{D}} [\ell(f_W(x), y)], \end{aligned}$$

denote the training (and test) losses of f_W , given n independent samples from data distribution $\mathcal{D}$. Let $\mathcal{Q}$ denote the *posterior* distribution. Specifically, we consider $\mathcal{Q}$ as being centered at the learned hypothesis W (which could be anywhere within the hypothesis space), given by a Gaussian distribution $\mathcal{N}(W, \Sigma)$, where Σ is a p by p covariance matrix. Given a sample $U \sim \mathcal{N}(0, \Sigma)$, let the perturbed loss be given by

$$\ell_{\mathcal{Q}}(f_W(x), y) = \mathbb{E}_U [\ell(f_{W+U}(x), y)]. \tag{8}$$

Then, let $\hat{L}_{\mathcal{Q}}(W)$ be the averaged value of $\ell_{\mathcal{Q}}(f_W(\cdot), \cdot)$, taken over n empirical samples from the training dataset. Likewise, let $L_{\mathcal{Q}}(W)$ be the population average of $\ell_{\mathcal{Q}}(f_W(\cdot), \cdot)$, in expectation over an unseen data sample from the underlying data distribution.

Having introduced the notations, we start with the PAC-Bayes bound (McAllester 2013), stated as follows.

Theorem 2. Suppose the loss $\ell(f_W(x), y)$ lies in a bounded range $[0, C]$ given any $x \in \mathcal{X}$ with label y . Let $\mathcal{P}$ and $\mathcal{Q}$ be prior and posterior distributions on the weights W . For any $\beta \in (0, 1)$ and $\delta \in (0, 1)$, with probability at least $1 - \delta$, we have:

$$L_{\mathcal{Q}}(W) \leq \frac{1}{\beta} \hat{L}_{\mathcal{Q}}(W) + \frac{C(KL(\mathcal{Q}|\mathcal{P}) + \log \frac{1}{\delta})}{2\beta(1 - \beta)n}. \tag{9}$$

This result provides flexibility in setting β . Our results will set β to balance the two terms. We will need the KL divergence between the prior $\mathcal{P}$ and the posterior $\mathcal{Q}$ in the PAC-Bayesian analysis. This is stated in the following classical result.

Proposition 3. Suppose $\mathcal{P} = N(X, \Sigma)$ and $\mathcal{Q} = N(Y, \Sigma)$ are both Gaussian distributions with mean vectors given by $X \in \mathbb{R}^p, Y \in \mathbb{R}^p$, and population covariance matrix $\Sigma \in \mathbb{R}^{p \times p}$. The KL divergence between $\mathcal{P}$ and $\mathcal{Q}$ is equal to

$$KL(\mathcal{Q}||\mathcal{P}) = \frac{1}{2}(X - Y)^\top \Sigma^{-1}(X - Y).$$

We are interested in the perturbed loss, $\ell_{\mathcal{Q}}(f_W(x), y)$, which is the expectation of $\ell(f_{W+U}(x), y)$ over U . Using a Taylor expansion, we find that

$$\ell_{\mathcal{Q}}(f_W(x), y) - \ell(f_W(x), y) \leq \langle \Sigma, \nabla^2 \ell(f_W(x), y) \rangle + \epsilon,$$

where Σ is the population covariance matrix of the perturbation, ∇^2 is the Hessian matrix with respect to the weights of f_W , and $\epsilon = C_1(\text{Tr}[\Sigma])^{3/2}$. See Lemma 4 for the complete statement of this result.

Based on the above expansion, next, we apply the PAC-Bayes bound from equation (9) to an L -layer transformer neural network f_W parameterized by W . We note that the KL divergence between the prior and posterior distributions, which are both Gaussian, is equal to $\langle \Sigma^{-1}, vv^\top \rangle$, where v is the difference between the initialized and the trained weights.

Next, we combine the above Hessian-based bound and KL divergence in the PAC-Bayes bound. Let ∇_+^2 denote the truncated Hessian matrix where we set the negative eigenvalues of ∇^2 to zero. We have that

$$\langle \Sigma, \nabla_W^2 [\ell(f_W(x), y)] \rangle \leq \langle \Sigma, \nabla_+^2 [\ell(f_W(x), y)] \rangle. \quad (10)$$

Substituting into (9), and minimizing over β and Σ , we will derive an upper bound on the generalization error (between $L(f_W)$ and $\hat{L}(f_W)$) equal to

$$\alpha := \sup_{W \in \mathcal{W}} \sup_{(x, y) \sim \mathcal{D}} \frac{\sqrt{W^\top [\tilde{\nabla}_+^2 \ell(f_W(x), y)] W}}{\sqrt{n}}, \quad (11)$$

where $\mathcal{W}$ is the support of the distribution from which the data is drawn.

We will use a Taylor expansion to bound the perturbed loss, as follows.

Claim 1. Let f_W be twice-differentiable, parameterized by weight vector $W \in \mathbb{R}^p$. Let $U \in \mathbb{R}^p$ be another vector with dimension p . For any W and U , the following identity holds

$$\ell(f_{W+U}(x), y) = \ell(f_W(x), y) + U^\top \nabla \ell(f_W(x), y) + \frac{1}{2} U^\top [\nabla^2 \ell(f_W(x), y)] U + R_2(\ell(f_W(x), y)),$$

where $R_2(\ell(f_W(x), y))$ is a second-order error term in a Taylor expansion of $\ell \circ f_W$ around W .

Proof. The proof follows from the fact that $\ell \circ f_W$ is twice-differentiable. By the mean value theorem, there must exist η between W and $U + W$ such that

$$R_2(\ell(f_W(x), y)) = U^\top \left(\nabla^2 [\ell(f_\eta(x), y)] - \nabla^2 [\ell(f_W(x), y)] \right) U.$$

This completes the proof of the claim.

Based on the above result, we provide a Taylor's expansion for $\ell_{\mathcal{Q}}$ minus ℓ .

Lemma 4. In the setting of Theorem 1, suppose each parameter is perturbed by an independent random variable drawn from $N(0, \Sigma)$. Let $\ell_{\mathcal{Q}}(f_W(x), y)$ be the loss averaged over the noise. There is a value C_1 that does not depend on n and $1/\delta$ such that

$$\ell_{\mathcal{Q}}(f_W(x), y) - \ell(f_W(x), y) \leq \frac{1}{2} \langle \Sigma, \nabla^2 [\ell(f_W(x), y)] \rangle + C_1 (\text{Tr}[\Sigma])^{\frac{3}{2}}. \quad (12)$$

Proof. We take the expectations over U of both sides of the equation in Claim 1. The result becomes

$$\mathbb{E}_U [\ell(f_{W+U}(x), y)] = \mathbb{E}_U \left[\ell(f_W(x), y) + U^\top \nabla \ell(f_W(x), y) + \frac{1}{2} U^\top \nabla^2 [\ell(f_W(x), y)] U + R_2(\ell(f_W(x), y)) \right].$$

Then, we use the perturbation distribution to calculate

$$\ell_{\mathcal{Q}}(f_W(x), y) = \mathbb{E}_U [\ell(f_W(x), y)] + \mathbb{E}_U [U^\top \nabla \ell(f_W(x), y)] + \frac{1}{2} \mathbb{E}_U [U^\top \nabla^2 [\ell(f_W(x), y)] U] + \mathbb{E}_U [R_2(\ell(f_W(x), y))].$$

Since $\mathbb{E}[U] = 0$, the first-order term vanishes. The second-order term becomes

$$\mathbb{E}_U [U^\top \nabla^2 [\ell(f_W(x), y)] U] = \langle \Sigma, \nabla^2 [\ell(f_W(x), y)] \rangle. \quad (13)$$

The expectation of the error term $R_2(\ell(f_W(x), y))$ be

$$\begin{aligned}\mathbb{E}_U [R_2(\ell(f_W(x), y))] &= \mathbb{E}_U [U^\top (\nabla^2[\ell(f_\eta(x), y)] - \nabla^2[\ell(f_W(x), y)])U] \\ &\leq \mathbb{E}_U [\|U\|_2^2 \cdot \|\nabla^2[\ell(f_\eta(x), y)] - \nabla^2[\ell(f_W(x), y)]\|_F] \\ &\lesssim \mathbb{E}_U [\|U\|_2^2 \cdot C_1 \|U\|_2] \\ &\lesssim C_1 (\mathbb{E} [U^\top U])^{\frac{3}{2}} = C_1 (\text{Tr} [\Sigma])^{\frac{3}{2}}.\end{aligned}$$

To obtain the first inequality in the last line, we have used that η lies on the line between W and $W + U$, so that $\|\eta\| \leq \|U\|$, and that the Hessian is Lipschitz. Thus, the proof is complete.

The last piece we will need is the uniform convergence of the Hessian, which uses the fact that the Hessian is Lipschitz continuous.

Lemma 5. *In the setting of Theorem 1, there exist values C_2, C_3 that do not grow with n and $1/\delta$, such that for any $\delta > 0$, with probability at least $1 - \delta$ over the randomness of the n training examples, we have*

$$\left\| \frac{1}{n} \sum_{i=1}^n \nabla^2[\ell(f_W(x_i), y_i)] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F \leq \frac{C_2 \sqrt{\log(C_3 n / \delta)}}{\sqrt{n}}. \quad (14)$$

The proof will be deferred to Section . With these results ready, we will provide proof of the Hessian-based generalization bound.

Now, we present the full proof of Theorem 1.

Proof of Theorem 1. First, we separate the gap between $L(W)$ and $\frac{1}{\beta} \hat{L}(W)$ into three parts:

$$L(W) - \frac{1}{\beta} \hat{L}(W) = L(W) - L_{\mathcal{Q}}(W) + L_{\mathcal{Q}}(W) - \frac{1}{\beta} \hat{L}_{\mathcal{Q}}(W) + \frac{1}{\beta} \hat{L}_{\mathcal{Q}}(W) - \frac{1}{\beta} \hat{L}(W).$$

By Lemma 4, we can bound the difference between $L(W)$ and $L_{\mathcal{Q}}(W)$ by the Hessian trace plus an error:

$$\begin{aligned}L(W) - \frac{1}{\beta} \hat{L}(W) &\leq - \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\frac{1}{2} \langle \Sigma, \nabla^2[\ell(f_W(x), y)] \rangle \right] + C_1 (\text{Tr} [\Sigma])^{\frac{3}{2}} + \left(L_{\mathcal{Q}}(W) - \frac{1}{\beta} \hat{L}_{\mathcal{Q}}(W) \right) \\ &\quad + \frac{1}{\beta} \left(\frac{1}{n} \sum_{i=1}^n \frac{1}{2} \langle \Sigma, \text{Tr} [\nabla^2[\ell(f_W(x_i), y_i)]] \rangle + C_1 (\text{Tr} [\Sigma])^{\frac{3}{2}} \right).\end{aligned}$$

After rearranging the terms, we find

$$\begin{aligned}L(W) - \frac{1}{\beta} \hat{L}(W) &\leq - \underbrace{\mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\frac{1}{2} \langle \Sigma, \nabla^2[\ell(f_W(x), y)] \rangle \right] + \frac{1}{n\beta} \sum_{i=1}^n \frac{1}{2} \langle \Sigma, \nabla^2[\ell(f_W(x_i), y_i)] \rangle}_{E_1} \\ &\quad + \underbrace{\frac{1+\beta}{\beta} C_1 \text{Tr} [\Sigma]^{\frac{3}{2}} + L_{\mathcal{Q}}(W) - \frac{1}{\beta} \hat{L}_{\mathcal{Q}}(W)}_{E_2}.\end{aligned} \quad (15)$$

We analyze E_1 by separating it into two parts

$$E_1 = \frac{1}{\beta} \left(\frac{1}{n} \sum_{i=1}^n \frac{1}{2} \langle \Sigma, \nabla^2[\ell(f_{\hat{W}}(x_i), y_i)] \rangle - \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[\frac{1}{2} \langle \Sigma, \nabla^2[\ell(f_W(x), y)] \rangle \right] \right) \quad (16)$$

$$+ \frac{1-\beta}{2\beta} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\langle \Sigma, \nabla^2 \ell(f_W(x), y) \rangle]. \quad (17)$$

We can use the uniform convergence result of Lemma 5 to bound the term in (16), leading to:

$$\begin{aligned}&\frac{1}{2\beta} \left(\frac{1}{n} \sum_{i=1}^n \langle \Sigma, \nabla^2 \ell(f_W(x_i), y_i) \rangle - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\langle \Sigma, \nabla^2 \ell(f_W(x), y) \rangle] \right) \\ &\leq \frac{\sigma^2}{2\beta} \cdot \sqrt{p} \cdot \left\| \frac{1}{n} \sum_{i=1}^n \nabla^2[\ell(f_W(x_i), y_i)] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F \quad (\text{by Cauchy-Schwarz}) \\ &\leq \frac{\sigma^2 \sqrt{p} \cdot C_2 \sqrt{\log(C_3 n / \delta)}}{2\beta \sqrt{n}}.\end{aligned} \quad (18)$$

In particular, the second step also uses the fact that the Hessian is a symmetric p by p matrix. As for equation (17), we have that

$$\frac{1-\beta}{2\beta} \mathbb{E}_{(x,y) \sim \mathcal{D}} [\langle \Sigma, \nabla^2 \ell(f_W(x), y) \rangle] \leq \frac{1-\beta}{2\beta} \langle \Sigma, \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla_+^2 \ell(f_W(x), y)] \rangle,$$

Combined with equation (18), we have shown that

$$E_1 \leq \frac{\sigma^2 \sqrt{p} \cdot C_2 \sqrt{\log(C_3 n / \delta)}}{2\beta \sqrt{n}} + \frac{1-\beta}{2\beta} \langle \Sigma, \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla_+^2 \ell(f_W(x), y)] \rangle. \quad (19)$$

Next, for E_2 , we use the PAC-Bayes bound of Theorem 2. In particular, we set the prior distribution $\mathcal{P}$ as the distribution of U and the posterior distribution $\mathcal{Q}$ as the distribution of $W + U$. Thus,

$$E_2 \leq \frac{C(KL(\mathcal{Q}||\mathcal{P}) + \log(\delta^{-1}))}{2\beta(1-\beta)n} \leq \frac{C\left(\frac{1}{2}W^\top \Sigma^{-1}W + \log(\delta^{-1})\right)}{2\beta(1-\beta)n}. \quad (20)$$

Combining equations (15), (19), (20), we claim that with probability at least $1 - 2\delta$, we must have:

$$L(W) - \frac{1}{\beta} \hat{L}(W) \leq \frac{\sigma^2 \sqrt{p} \cdot C_2 \sqrt{\log(C_3 n / \delta)}}{2\beta \sqrt{n}} + \frac{1+\beta}{\beta} C_1 \text{Tr}[\Sigma]^{3/2} + \frac{C \log \frac{1}{\delta}}{2\beta(1-\beta)n} \quad (21)$$

$$+ \frac{CW^\top \Sigma^{-1}W}{4\beta(1-\beta)n} + \frac{1-\beta}{2\beta} \langle \Sigma, \mathbf{H}_W \rangle, \quad (22)$$

where

$$\mathbf{H}_W = \mathbb{E}_{(x,y) \sim \mathcal{D}} [[\nabla^2 \ell(f_W(x), y)]^+].$$

We next choose Σ and $\beta \in (0, 1)$ to minimize the above bound. In particular, we set

$$\Sigma = \sqrt{\frac{C}{2(1-\beta)^2 n \|W\|^2}} \mathbf{H}_W^{-\frac{1}{2}} W W^\top \quad (23)$$

so that the two terms in equation (22) are equal to each other:

$$\begin{aligned} \frac{1-\beta}{\beta} \cdot \sqrt{\frac{C}{2(1-\beta)^2 n \|W\|^2}} \langle \mathbf{H}_W^{\frac{1}{2}} W, W \rangle &\leq \frac{1}{\beta} \sqrt{\frac{C}{2n \|W\|^2}} \|\mathbf{H}_W^{\frac{1}{2}} W\| \|W\| \\ &= \frac{1}{\beta} \sqrt{\frac{C}{2n}} \|W^\top \mathbf{H}_W W\| \\ &\leq \frac{1}{\beta} \sqrt{\frac{C\mathcal{H}}{2n}}. \end{aligned}$$

By plugging in σ to equation (21) and re-arranging terms, the gap between $L(W)$ and $\beta^{-1} \hat{L}(W)$ can be bounded as:

$$L(W) - \frac{1}{\beta} \hat{L}(W) \leq \frac{1}{\beta} \sqrt{\frac{C\mathcal{H}}{n}} + \frac{C_2 \sqrt{2p \log(C_3 n / \delta)}}{2\beta \sqrt{n}} \sigma^2 + \frac{1+\beta}{\beta} C_1 \text{Tr}[\Sigma]^{3/2} + \frac{C}{2\beta(1-\beta)n} \log \frac{1}{\delta}.$$

Let $\beta = 1/(1+\epsilon)$ and so that $\epsilon = (1-\beta)/\beta$. We find that

$$\begin{aligned} L(W) &\leq (1+\epsilon) \hat{L}(W) + (1+\epsilon) \sqrt{\frac{C\mathcal{H}}{n}} + \xi, \text{ where} \\ \xi &= \frac{C_2 \sqrt{2p \log(C_3 n / \delta)}}{2\beta \sqrt{n}} \sigma^2 + \left(1 + \frac{1}{\beta}\right) C_1 \text{Tr}[\Sigma]^{3/2} + \frac{C}{2\beta(1-\beta)n} \log \frac{1}{\delta}. \end{aligned}$$

Notice that ξ is of order

$$O(n^{-\frac{3}{4}} + n^{-\frac{3}{4}} + \log(\delta^{-1})n^{-1}) = O(\log(\delta^{-1})n^{-\frac{3}{4}}).$$

Therefore, we have shown that with probability at least $1 - \delta$,

$$L(f_W) \leq (1+\epsilon) \hat{L}(f_W) + (1+\epsilon) \sqrt{\frac{C\mathcal{H}}{n}} + O(n^{-3/4}). \quad (24)$$

The proof is now complete.

Proof of Lemma 5 In this section, we provide the proof of Lemma 5, which shows the uniform convergence of the loss Hessian.

Proof of Lemma 5. Let $C, \epsilon > 0$, and let

$$S = \{W \in \mathbb{R}^p : \|W\|_2 \leq C\}.$$

There exists an ϵ -cover of S with respect to the ℓ_2 -norm with at most $\max\left(\left(\frac{3C}{\epsilon}\right)^p, 1\right)$ elements; see, e.g., Example 5.8 (Wainwright 2019a). Let $T \subseteq S$ denote this cover. Recall that the Hessian $\nabla^2[\ell(f_W(x), y)]$ is C_1 -Lipschitz for all $(W + U) \in S, W \in S$. Then we have

$$\|\nabla^2[\ell(f_{W+U}(x), y)] - \nabla^2[\ell(f_W(x), y)]\|_F \leq C_1 \|U\|_2.$$

For $\delta, \epsilon > 0$, define the event

$$E = \left\{ \forall W \in T, \left\| \frac{1}{n} \sum_{i=1}^n \nabla^2[\ell(f_W(x_i), y_i)] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F \leq \delta \right\}.$$

By the matrix Bernstein inequality (Vershynin 2018), we have

$$\Pr[E] \geq 1 - 4 \cdot |\mathcal{N}| \cdot p \cdot \exp\left(-\frac{n\delta^2}{2\alpha^2}\right). \quad (25)$$

Next, for any $W \in S$, we can pick some $W + U \in T$ such that $\|U\|_2 \leq \epsilon$. We have

$$\begin{aligned} \left\| \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_{W+U}(x), y)]] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F &\leq C_1 \|U\|_2 \leq C_1 \epsilon \\ \left\| \frac{1}{n} \sum_{j=1}^n \nabla^2[\ell(f_{W+U}(x_j), y_j)] - \frac{1}{n} \sum_{j=1}^n \nabla^2[\ell(f_W(x_j), y_j)] \right\|_F &\leq C_1 \|U\|_2 \leq C_1 \epsilon. \end{aligned}$$

Therefore, for any $W \in S$, we obtain:

$$\left\| \frac{1}{n} \sum_{j=1}^n \nabla^2[\ell(f_W(x_j), y_j)] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F \leq 2C_1 \epsilon + \delta.$$

Set $\epsilon = \delta/(2C_1)$ so that on the event E ,

$$\left\| \frac{1}{n} \sum_{j=1}^n \nabla^2[\ell(f_W(x_j), y_j)] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F \leq 2\delta.$$

The event E happens with a probability of at least:

$$1 - 4|T|p \cdot \exp\left(-\frac{n\delta^2}{2\alpha^2}\right) = 1 - 4p \cdot \exp\left(\log |T| - \frac{n\delta^2}{2\alpha^2}\right).$$

Now, we have $\log |T| \leq p \log(3B/\epsilon) = p \log(6CC_1/\delta)$. If we set

$$\delta = \sqrt{\frac{4p\alpha^2 \log(3\tau CC_1 n/\alpha)}{n}}, \quad (26)$$

so that $\log(3\tau CC_1 n/\alpha) \geq 1$ (because $n \geq \frac{\epsilon\alpha}{3C_1}$ and $\tau \geq 1$), then we find

$$\begin{aligned} p \log(6CC_1/\delta) - n\delta^2/(2\alpha^2) &= p \log\left(\frac{6CC_1\sqrt{n}}{\sqrt{4p\alpha^2 \log(3\tau CC_1 n/\alpha)}}\right) - 2p \log(3\tau CC_1 n/\alpha) \\ &= p \log\left(\frac{3CC_1\sqrt{n}}{\alpha\sqrt{p \log(3\tau CC_1 n/\alpha)}}\right) - 2p \log(3\tau CC_1 n/\alpha) \\ &\leq p \log(3\tau CC_1 n/\alpha) - 2p \log(3\tau CC_1 n/\alpha) \quad (\tau \geq 1, \log(3\tau CC_1 n/\alpha) \geq 1) \\ &= -p \log(3\tau CC_1 n/\alpha) \leq -p \log(e\tau). \quad (3CC_1 n/\alpha \geq e) \end{aligned}$$

Environment	Goal	State	Action	Reward
MT10	Execute robotic manipulation tasks	$\mathbb{R}^{39}$, representing the state of the robot and the target objective.	4 continuous actions: controlling end-effector movement and gripper actuation.	A sparse reward for moving the objective to its goal position.
CartPole	Balance a pole attached to a cart	$\mathbb{R}^4$, representing the cart's position, velocity, the pole's angle, and angular velocity.	2 discrete actions: applying a push to the cart (left or right).	A reward of +1 for each timestep the pole remains upright.
Highway	Avoid collisions while driving on a highway	$\mathbb{R}^{5 \times 5}$, representing the kinematic states of the ego vehicle and four nearby vehicles.	5 discrete actions: change lane left/right, maintain lane, accelerate, decelerate.	A dense reward for maintaining high speed, with a large penalty for collisions.
LunarLander	Land a spacecraft safely on the ground	$\mathbb{R}^8$, representing the lander's coordinates, velocities, angle, and leg contact status.	4 discrete actions: control the main engine and side thrusters.	A shaped reward for approaching the landing pad, plus a large bonus for a soft landing.

Table 4: A summary of RL environments tested in our experiments.

Therefore, with a probability at least

$$1 - 4|\mathcal{N}|p \cdot \exp(-n\delta^2/(2\alpha^2)) \geq 1 - 4p(e\tau)^{-p}, \quad (27)$$

we have

$$\left\| \frac{1}{n} \sum_{j=1}^n \nabla^2[\ell(f_W(x_j), y_j)] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F \leq \sqrt{\frac{16p\alpha^2 \log(3\tau CC_1 n/\alpha)}{n}}.$$

Denote $\delta' = 4p(e\tau)^{-p}$, $C_2 = 4\alpha\sqrt{p}$, and $C_3 = 12pCC_1/(e\alpha)$. With probability at least $1 - \delta'$, we have

$$\left\| \frac{1}{n} \sum_{i=1}^n \nabla^2[\ell(f_W(x_i), y_i)] - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\nabla^2[\ell(f_W(x), y)]] \right\|_F \leq C_2 \sqrt{\frac{\log(C_3 n/\delta')}{n}}.$$

This completes the proof of Lemma 5.

Sample complexities for Q -learning. The minimax optimal sample complexity to find an ϵ -optimal Q -function in tabular γ -discounted MDPs is known to be $\tilde{O}(SA(1-\gamma)^{-3}\epsilon^{-2})$ (Chi, Chen, and Wei 2025). This rate is achieved by model-based algorithms. Standard synchronous Q -learning, however, is sub-optimal, requiring $\tilde{O}(SA(1-\gamma)^{-4}\epsilon^{-2})$ samples. This gap in horizon dependency was later closed by variance reduction techniques. For instance, a variance-reduced Q -learning algorithm (Wainwright 2019b) achieves the minimax optimal rate, matching the lower bound up to logarithmic factors. See a recent monograph for extensive references (Foster and Rakhlin 2023). It is an interesting open question to study the sample complexity of Q -learning for multi-objective reinforcement learning. This is left for future work.

Omitted Experimentals

Environments. We summarize the RL environments: MT10, CartPole, Highway, LunarLander in Table 4. We adopt the Gymnasium library (Towers et al. 2025) for environment implementation.

RL algorithms. In the Meta-World experiments, we employ Soft Actor-Critic (SAC) (Haarnoja et al. 2018) as the policy gradient algorithm. SAC is an off-policy, actor-critic deep reinforcement learning algorithm based on the maximum entropy reinforcement learning framework. The core idea is to encourage exploration by maximizing a trade-off between the expected return and the policy's entropy.

The SAC algorithm learns three functions: A policy π_θ and two critic functions Q_{ψ_1} and Q_{ψ_2} . We train the critic networks by minimizing the mean squared Bellman error:

$$L(\psi_i) = \hat{\mathbb{E}}_{(s_t, a_t, r_t, s_{t+1}, d_t) \sim \mathcal{D}} \left[\frac{1}{2} (Q_{\psi_i}(s_t, a_t) - y(r_t, s_{t+1}, d_t))^2 \right], \quad (28)$$

The target value y is shared between both critics and is computed as:

$$y(r_t, s_{t+1}, d_t) = r_t + \gamma(1 - d_t) \left(\min_{j=1,2} Q_{\psi_{j,\text{target}}}(s_{t+1}, a_{t+1}) - \alpha \log \pi_{\theta}(a_{t+1} | s_{t+1}) \right), \text{ with } a_{t+1} \sim \pi_{\theta}(\cdot | s_{t+1}). \quad (29)$$

The target network parameters $\psi_{j,\text{target}}$ is updated by

$$\psi_{j,\text{target}} \leftarrow \tau \psi_j + (1 - \tau) \psi_{j,\text{target}}.$$

Next, the actor parameters θ are trained to maximize the policy’s expected return and entropy:

$$L_{\text{SAC}}(\theta) = \mathbb{E}_{\tilde{s}_t \sim \mathcal{D}} \left[\mathbb{E}_{a_t \sim \pi_{\theta}(\cdot | s_t)} \left[\alpha \log \pi_{\theta}(a_t | s_t) - \min_{j=1,2} Q_{\psi_j}(s_t, a_t) \right] \right] \quad (30)$$

In the control environments, we employ Proximal Policy Optimization (PPO) (Schulman et al. 2017) as our policy gradient algorithm. PPO follows the design of the actor-critic algorithm (Konda and Tsitsiklis 1999), where the core optimization objective is the clipped surrogate policy loss. This is combined with a value function loss and an entropy as the optimization objective.

PPO involves tackling the following objective:

$$L_{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] - c_1 \mathbb{E}_t \left[(V_{\theta}(s_t) - \hat{V}_t)^2 \right] + c_2 \mathbb{E}_t [\mathcal{H}[\pi_{\theta}](s_t)], \quad (31)$$

where

$$r_t(\theta_t) = \frac{\pi_{\theta_t}(a_t | s_t)}{\pi_{\theta_{t-1}}(a_t | s_t)}$$

is the probability ratio between the new policy and the previous policy, $\hat{A}_t$ is the advantage estimation computed from the value function $V_{\theta}(s_t)$, and $\mathcal{H}[\pi_{\theta}](s_t)$ is the entropy of the policy.

In the meta-RL setting, we use the MAML algorithm (Finn, Abbeel, and Levine 2017), which consists of a two-step optimization process. In the inner loop adaptation, MAML computes an adapted policy for each source task T_i from the trajectory $\mathcal{D}_{\theta}^i$ sampled by the interaction with the meta policy θ and the corresponding environment:

$$\theta'_i = \theta - \alpha \nabla_{\theta} L_{T_i}(\pi_{\theta}). \quad (32)$$

In the outer loop meta-update, MAML then aggregates the data from the trajectory sampled by the interaction with the task-specific policy ϕ_1^i and the corresponding environment:

$$\theta \leftarrow \theta + \beta \cdot \nabla_{\theta} \sum_{T_i \in \mathcal{T}} L_{T_i}(\pi_{\theta}). \quad (33)$$

Baseline descriptions. PaCo (Parameter Compositional MTRL) introduces a highly flexible framework where task-specific parameters are explicitly composed rather than separated (Sun et al. 2022). In PaCo, the parameter vector for a given task is constructed as a linear combination of a shared, task-agnostic basis of parameter vectors and a learned, task-specific compositional vector. Other notable architectures, such as Soft Modularization (SM) and those based on Mixture-of-Experts (MoE), explore ways to dynamically route information and learn task-specific components in a unified network (Yang et al. 2020; Sun et al. 2022).

Simultaneous optimization of a shared set of parameters under competing demands of various tasks frequently leads to destructive interference, where joint training results in worse performance on a given task than training it alone (Yu et al. 2020a; Teh et al. 2017). The gradients from different tasks can point in conflicting directions, which destabilizes learning (Yu et al. 2020a; Liu et al. 2021).

Implementation details. In the Meta-World benchmark, we use training steps of 10^7 , a buffer size of 10^6 , a batch size of 1280, and a learning rate of 3×10^{-4} with the Adam optimizer.

In the CartPole, Highway, and LunarLander environments, we use the following settings: 100 meta-training steps, 2048 training steps for each inner loop policy update, and a batch size of 64.

Additional experiments. We find that the shifted distribution of the source task $\mathcal{T}$ affects the generalization on the unseen task. We conduct a straightforward experiment in a 2D navigation environment to empirically investigate the influence of the source task distribution on meta reinforcement learning. In this environment, the agent starts at the origin $(0, 0)$ and navigates towards a goal within a $[-1, 1] \times [-1, 1]$ plane. All goals are located on a circle centered at the origin with a fixed radius of 0.9. An episode is considered a success if the agent’s final distance to the goal is within a threshold of 0.1. The agent is guided by a reward function defined as the negative Euclidean distance to the goal, with an additional bonus upon success.

For the experiment, we maintain a uniform distribution for the target tasks, where goals can appear anywhere on the circle. We then analyze the agent’s adaptation performance after being meta-trained on source tasks drawn from three different distributions. We find that:

1. In Figure 3b, when the source task distribution is uniform and covers the four directions of the target space, the agent can easily adapt to any target task.
2. In Figure 3c, when the source tasks do not cover the target tasks, the policy overfits to the source distribution and fails to generalize.
3. In Figure 3d, when the source distribution covers the target space but is heavily biased, the policy struggles to adapt to target tasks in the sparsely represented regions.

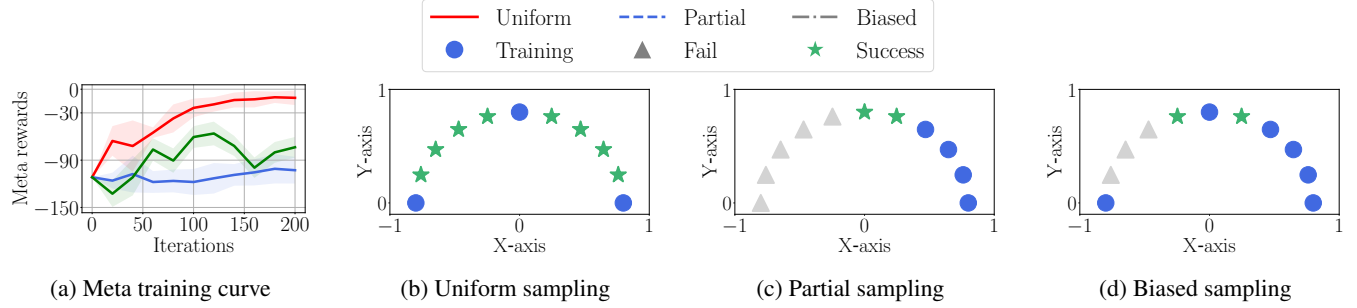


Figure 3: We illustrate the meta-training curve and the corresponding meta task distribution of the navigation environment. The agent is meta-trained to navigate a training set of goals, and then tested on a distinct set of unseen test goals with a few adaptation steps.